

NLA_Agent 项目进度汇总

江斐阳

2026 年 4 月 20 日

1 项目介绍

NLA_Agent 的核心目标是面向数值线性代数计算场景，探索一种以自然语言交互为核心、能够在部分典型任务上替代 MATLAB 的智能 Agent 系统。与传统 MATLAB 主要依赖命令输入、函数调用和脚本编写的使用方式不同，本项目尝试将大语言模型的语义理解能力与数值计算后端结合起来，使用户能够通过自然语言完成问题描述、任务表达与结果获取。

本项目的研究出发点主要体现在以下几个方面：

- MATLAB 在数值计算领域具有成熟生态，但其交互方式对初学者存在一定门槛，尤其体现在函数记忆、参数组织和脚本编写方面；
- 大语言模型在自然语言理解、任务拆解与交互式推理方面已经展现出较强潜力，为构建智能数学计算系统提供了新的技术基础；
- 数值线性代数具有问题类型相对明确、算法体系较成熟、验证方式较清晰等特点，适合作为智能 Agent 替代 MATLAB 的切入场景；
- 相比传统 MATLAB，本项目不仅希望实现“完成计算”，还希望系统具备“理解问题、提示前提、推荐算法、积累经验”的能力。

基于上述考虑，本项目计划首先聚焦于数值线性代数典型任务，构建一个可运行、可扩展、可验证的原型框架。现阶段重点验证的问题包括：

- 系统能否正确理解用户以自然语言提出的数值线性代数问题；
- 系统能否完成任务识别、前提检查、矩阵分析、算法推荐和数值求解；
- 系统能否逐步形成接近 MATLAB 的交互式使用体验；
- 系统能否在此基础上进一步引入记忆、检索和解释能力，形成区别于传统计算工具的智能化特征。

2 项目结构

当前项目结构主要包括以下几个部分：

- **Agent 主入口模块**：负责 Root Agent 的组装、模型接入、工具注册与总流程调度；
- **策略模块**：负责任务识别、用户偏好判断、必要条件检查、矩阵性质分析与算法推荐；
- **矩阵解析模块**：负责读取和转换不同格式的矩阵输入数据，如 CSC 格式与 Matrix Market 格式；
- **数值计算后端模块**：依托 NumPy、SciPy 及 LAPACK/BLAS 等库实现线性代数核心计算；
- **执行模块**：用于在必要场景下运行 Python 代码，以补充后端工具未覆盖的能力；
- **长期记忆模块**：负责历史问答经验的结构化抽取、向量检索、去重、合并和版本管理；
- **工作区模块**：用于保存变量、中间结果和 `ans` 等内容，以模拟 MATLAB 的交互式工作区机制；
- **测试模块**：用于验证部分关键功能模块的正确性和稳定性；
- **前端展示模块**：用于展示 Agent 输出内容，并支持 Markdown 与数学公式渲染。

3 已实现功能

在整体处理逻辑上，MLA_Agent 会尽量按照“目标澄清、前提检查、矩阵性质确认、求解规划、数值执行”的顺序推进问题处理。

在数值线性代数核心任务上，依托于 NumPy、SciPy、matplotlib 等 python 库，MLA_Agent 可以完成以下任务：

- 线性方程组求解；
- 最小二乘问题求解；
- 特征值与特征向量相关任务的识别与处理框架；
- 矩阵性质分析与算法推荐；
- 画出用户指定的图像。

在输入支持方面, NLA_Agent 目前已经实现了多种矩阵输入形式, 以适应更贴近实际的数据使用场景。当前已支持的输入方式包括:

- 在对话中直接输入矩阵内容;
- 读取 CSC 格式矩阵文件;
- 读取 Matrix Market 压缩格式矩阵文件。

在 MATLAB 替代目标相关的功能上, NLA_Agent 已经初步引入了工作区机制 (该机制仍处于基础阶段), 主要包括:

- 变量的保存与读取;
- 工作区变量列表查看;
- 工作区变量清理;

此外, NLA_Agent 还包含一个长期记忆模块, 用于支持历史问题经验的沉淀与复用。目前该模块已经具备以下能力:

- 将历史问答抽取为结构化记忆条目;
- 对新问题进行相似记忆检索;
- 对重复或近似记忆进行去重与合并;
- 对记忆条目进行版本维护和质量控制。

4 NLA_Agent 解决实际问题的典型例子

4.1 求解线性方程组

问题: 构造一个 20 阶的希尔伯特矩阵 A , 右端向量为该矩阵乘全一向量, 求解这个线性方程组, 要求对比使用直接求解方法和 svd 方法。

NLA_Agent 的解答: 我已经完成了 20 阶希尔伯特矩阵线性方程组的求解, 并对直接求解方法和 SVD 方法进行了详细对比。以下是关键发现:

1. 希尔伯特矩阵的性质

- **严重病态:** 条件数高达 1.32×10^{18} , 是数值计算中著名的病态矩阵。
- **数值秩不足:** 20 阶矩阵的数值秩只有 13, 说明矩阵接近奇异。
- **奇异值分布:** 奇异值从 1.91 迅速衰减到 1.45×10^{-18} , 跨越 18 个数量级。

2. 直接求解方法 (LAPACK gesv)

- **结果：**解向量分量在 10^3 到 10^{11} 量级，与真实解（全 1 向量）相差巨大。
- **残差：** 2.77×10^{-1} ，相对较大。
- **误差：**与真实解的误差高达 1.19×10^{12} 。
- **问题：**由于矩阵病态，直接求解方法（LU 分解）产生了严重的数值不稳定。

3. SVD 方法

- **SVD 最小二乘 (numpy.linalg.lstsq)**
 - **结果：**解向量非常接近全 1 向量（分量在 0.9999–1.0001 之间）。
 - **残差：** 2.81×10^{-15} ，接近机器精度。
 - **误差：** 1.11×10^{-3} ，比直接求解好 15 个数量级。
 - **优势：**自动处理秩亏问题，数值稳定。
- **截断 SVD (手动设置阈值)**
 - **结果：**解向量同样接近全 1 向量。
 - **残差：** 5.96×10^{-15} 。
 - **误差：** 2.83×10^{-3} 。
 - **优势：**可以显式控制截断阈值，平衡精度与稳定性。

方法	残差范数	解误差范数	解向量范围	数值稳定性
直接求解	2.77×10^{-1}	1.19×10^{12}	$[-6 \times 10^{11}, 7 \times 10^{11}]$	差
SVD 最小二乘	2.81×10^{-15}	1.11×10^{-3}	[1.00, 1.00]	优秀
截断 SVD	5.96×10^{-15}	2.83×10^{-3}	[1.00, 1.00]	优秀

结论与建议：对于病态矩阵，SVD 方法明显优于直接求解方法。SVD 的优势在于它可以通过奇异值分解显式处理矩阵的秩亏问题，并且能够通过截断小奇异值提高数值稳定性，因此对病态问题具有更强的鲁棒性。相比之下，直接求解对条件数非常敏感，容易产生严重的数值不稳定，尤其是对于接近奇异的矩阵，可能得到完全失真的结果。

实际应用中，当矩阵条件数大于 10^8 时，优先考虑 SVD 方法；必要时可以使用截断 SVD 来平衡精度与稳定性。对于希尔伯特矩阵这类特殊的病态问题，还应考虑正则化方法。

这个案例清晰地展示了为什么希尔伯特矩阵是数值线性代数中测试算法稳定性的经典例子，也说明了 SVD 方法在处理病态问题时的优越性。

5 目前存在的问题与后续计划

- 目前 NLA_Agent 对部分数值代数算法的使用能力较弱 (比如 Jacobi SVD), 存在一些调用错误函数的情况。后续计划通过手动分类让 Agent 调用正确的 numpy/scipy 函数。
- Agent 运行时在部分情况下仍可能会抛出 json 格式错误问题, 需要进一步严格控制 Agent 的 json 输出格式。
- 上下文过长时, Agent 的表现会下降。也许可以通过适当的上下文压缩缓解。
- 目前搭建的记忆库还较为简陋, 模型从问题中存储的记忆往往用处不大。
- 如果使用 google-adk 的 web 页面作为前端, 则不能支持 latex 公式的正常显示。目前自己搭建的前端虽然能支持渲染 latex 公式, 但是其余功能还不太齐全 (比如查看过往聊天记录、显示 Agent 的工具调用流程等)。
- 在交互体验方面, 也许可以仿照 MATLAB 在前端设置变量区, 方便用于查看目前已生成的矩阵和向量。
- NLA_Agent 目前使用 DeepSeek-V3.2 作为 “大脑”, 后续可以尝试使用其他大语言模型。